



FACULDADE CESAR SCHOOL

CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

**PROJETO DA DISCIPLINA TEORIA DA COMPUTAÇÃO: TEORIA DA
COMPLEXIDADE E ANÁLISE DE TEMPO DE ALGORITMOS (COUNTING SORT)**

ARTHUR XAVIER

GUSTAVO OLIVEIRA

JOSÉ VITOR

TALITA FRAGA

RECIFE

JUNHO/2026

SUMÁRIO

SUMÁRIO.....	2
1 - DESCRIÇÃO DO ALGORITMO.....	3
1.1 - PROBLEMA RESOLVIDO.....	3
1.2 - LÓGICA GERAL DO ALGORITMO.....	4
1.2.1 - CONTAGEM DAS OCORRÊNCIAS:.....	4
1.2.2 - SOMA ACUMULADA:.....	4
1.2.3 - CONSTRUÇÃO DO VETOR ORDENADO:.....	4
1.2.4 - PSEUDOCÓDIGO:.....	5
2 - CLASSIFICAÇÃO ASSINTÓTICA:.....	6
2.1 - COMPLEXIDADE BIG-O.....	6
2.2 - COMPLEXIDADE BIG-Ω.....	7
2.3 - COMPLEXIDADE BIG-Θ.....	7
2.4 - COMPLEXIDADE ESPACIAL.....	7
2.5 - ANÁLISE DOS CASOS.....	8
3 - DISCUSSÃO SOBRE APLICABILIDADE.....	9
3.1 - CONTEXTOS DE EFICIÊNCIA.....	9
3.2 - LIMITAÇÕES DO ALGORITMO.....	10
4 - METODOLOGIA EXPERIMENTAL.....	11
4.1 - AMBIENTE DE EXECUÇÃO.....	12
5 - RESULTADOS EXPERIMENTAIS.....	13
5.1 - VERIFICAÇÃO DA CORRETEDE DAS IMPLEMENTAÇÕES.....	13
5.2 - COMPARAÇÃO DOS TEMPOS DE EXECUÇÃO.....	14
5.3 - ADERÊNCIA DOS RESULTADOS À COMPLEXIDADE TEÓRICA.....	16
5.4 - COMPARAÇÃO ENTRE PYTHON E RUST.....	18
5.5 - TAXA DE PROCESSAMENTO.....	19
6 - REFLEXÃO FINAL SOBRE COMPLEXIDADE.....	20
7 - CONCLUSÃO.....	21

1 - DESCRIÇÃO DO ALGORITMO

O Counting Sort é um algoritmo de ordenação não-comparativo desenvolvido para organizar conjuntos de números inteiros de maneira eficiente. Diferentemente dos algoritmos clássicos de ordenação, como Quick Sort, Merge Sort e Heap Sort, o Counting Sort não realiza comparações diretas entre os elementos do vetor. Seu funcionamento baseia-se na contagem da frequência de ocorrência de cada valor presente na entrada.

A principal característica desse algoritmo é explorar um intervalo conhecido dos valores para realizar a ordenação em tempo linear. Dessa forma, o Counting Sort apresenta excelente desempenho em situações nas quais os dados possuem um domínio relativamente pequeno quando comparado à quantidade total de elementos.

Por utilizar estruturas auxiliares para armazenar contagens e posições acumuladas, o algoritmo consegue determinar diretamente a posição correta de cada elemento no vetor ordenado, evitando múltiplas trocas e comparações sucessivas.

1.1 - PROBLEMA RESOLVIDO

O problema abordado pelo Counting Sort consiste na ordenação de elementos inteiros presentes em um vetor desordenado. O objetivo é reorganizar os valores em ordem crescente de forma eficiente, minimizando o custo computacional associado ao processo de ordenação.

Exemplo:

Entrada: [4, 2, 2, 8, 3, 3, 1]

Saída: [1, 2, 2, 3, 3, 4, 8]

Nesse caso, o algoritmo deve identificar quantas vezes cada valor aparece e, a partir dessas informações, construir o vetor ordenado.

1.2 - LÓGICA GERAL DO ALGORITMO

O funcionamento do Counting Sort pode ser dividido em três etapas principais.

1.2.1 - CONTAGEM DAS OCORRÊNCIAS:

Inicialmente , o algoritmo percorre o vetor original e registra a quantidade de vezes que cada valor aparece. Essas informações são armazenadas em um vetor auxiliar denominado vetor de contagem.

Exemplo:

Vetor Original: [4, 2, 2, 8, 3, 3, 1]

Vetor de Contagem:

Índice: 0 1 2 3 4 5 6 7 8

Contagem: 0 1 2 2 1 0 0 0 1

Cada posição do vetor representa a frequência do valor correspondente ao índice.

1.2.2 - SOMA ACUMULADA:

Após a contagem, o vetor auxiliar é transformado em um vetor de soma acumulada. Nessa etapa, cada posição passa a armazenar a quantidade total de elementos menores ou iguais ao índice correspondente.

Exemplo: [0, 1, 3, 5, 6, 6, 6, 6, 7]

Esse processo permite determinar a posição correta de cada elemento do vetor final.

1.2.3 - CONSTRUÇÃO DO VETOR ORDENADO:

Na última etapa, o algoritmo percorre o vetor original da direita para a esquerda e posiciona cada elemento em sua posição correta no vetor de saída. Para isso, utiliza o vetor de soma acumulada, que indica a posição final de cada valor no resultado ordenado.

A cada elemento processado, o algoritmo consulta sua posição no vetor de contagem acumulada, insere o valor no vetor de saída e decrementa a contagem correspondente. Esse processo garante que todos os elementos sejam posicionados corretamente.

Ao final da execução, para o exemplo apresentado, obtém-se o vetor ordenado:

[1, 2, 2, 3, 3, 4, 8]

Quando implementado da direita para a esquerda, o algoritmo mantém a ordem relativa entre elementos iguais, característica conhecida como estabilidade.

1.2.4 - PSEUDOCÓDIGO:

COUNTING-SORT(A)

1. maior $\leftarrow$ maior elemento de A
2. criar vetor CONTAGEM[0..maior]
3. inicializar CONTAGEM com zero
4. para cada elemento x em A:
 $\text{CONTAGEM}[x] \leftarrow \text{CONTAGEM}[x] + 1$
5. para i de 1 até maior:
 $\text{CONTAGEM}[i] \leftarrow \text{CONTAGEM}[i] + \text{CONTAGEM}[i - 1]$
6. criar vetor SAÍDA
7. para cada elemento x de A da direita para esquerda:

$\text{CONTAGEM}[x] \leftarrow \text{CONTAGEM}[x] - 1$

$\text{SAÍDA}[\text{CONTAGEM}[x]] \leftarrow x$

8. retornar SAÍDA

2 - CLASSIFICAÇÃO ASSINTÓTICA:

A análise assintótica do Counting Sort considera tanto a quantidade de elementos do vetor quanto o intervalo dos valores armazenados.

Sejam:

- n o número de elementos do vetor;
- k o maior valor possível presente na entrada.

O algoritmo executa operações lineares sobre essas duas quantidades.

2.1 - COMPLEXIDADE BIG-O

No pior caso, o Counting Sort percorre:

- o vetor original para realizar a contagem;
- o vetor auxiliar para calcular a soma acumulada;
- o vetor novamente para construir a saída.

Dessa forma, sua complexidade assintótica é:

$O(n+k)$

Isso significa que o tempo de execução cresce linearmente em relação ao número de elementos e ao intervalo dos valores.

2.2 - COMPLEXIDADE BIG-Ω

No melhor caso, o algoritmo ainda precisa percorrer o vetor de entrada e o vetor auxiliar ao menos uma vez. Portanto, sua complexidade mínima permanece:

$$\Omega(n+k)$$

Isso ocorre porque o Counting Sort não depende da ordem inicial dos elementos.

2.3 - COMPLEXIDADE BIG-Θ

Como o comportamento do algoritmo permanece linear tanto no melhor quanto no pior caso, sua complexidade assintótica exata é:

$$\Theta(n+k)$$

Assim, o Counting Sort mantém desempenho previsível independentemente da disposição inicial dos dados.

2.4 - COMPLEXIDADE ESPACIAL

Além da complexidade temporal, o Counting Sort utiliza estruturas auxiliares para realizar a ordenação.

O algoritmo necessita de um vetor de contagem com tamanho proporcional ao intervalo dos valores possíveis (k) e de um vetor de saída com tamanho proporcional ao número de elementos da entrada (n).

Dessa forma, sua complexidade espacial é dada por:

$$O(n + k)$$

Isso significa que o consumo de memória cresce tanto em função da quantidade de elementos quanto do intervalo dos valores possíveis. Em situações nas quais k é muito maior que n , o custo de memória pode se tornar uma limitação importante para a utilização do algoritmo.

2.5 - ANÁLISE DOS CASOS

Diferentemente dos algoritmos de ordenação baseados em comparação, o desempenho do Counting Sort não depende da ordem inicial dos elementos presentes na entrada. Independentemente do vetor estar ordenado, invertido ou distribuído aleatoriamente, o algoritmo sempre executa as mesmas etapas: contagem das ocorrências, cálculo da soma acumulada e construção do vetor de saída.

Melhor Caso:

No melhor caso, o algoritmo percorre o vetor de entrada para realizar a contagem das ocorrências, percorre o vetor auxiliar para calcular a soma acumulada e constrói o vetor ordenado. Assim, sua complexidade permanece:

$$\Theta(n + k)$$

Caso Médio:

Para entradas aleatórias, o comportamento do algoritmo não sofre alterações significativas, pois o número de operações realizadas continua dependente apenas de n e k . Portanto, sua complexidade é:

$$\Theta(n + k)$$

Pior Caso:

Mesmo em situações desfavoráveis, o Counting Sort continua executando as mesmas etapas e realizando a mesma quantidade de operações. Dessa forma, sua complexidade permanece:

$$\Theta(n + k)$$

Portanto, o Counting Sort apresenta comportamento previsível e uniforme, mantendo a mesma complexidade assintótica nos três cenários analisados.

3 - DISCUSSÃO SOBRE APLICABILIDADE

3.1 - CONTEXTOS DE EFICIÊNCIA

O Counting Sort apresenta excelente desempenho em cenários específicos, principalmente quando:

- os dados são compostos por números inteiros;
- o intervalo dos valores é pequeno;
- há necessidade de ordenação rápida;
- deseja-se estabilidade na ordenação.

Nessas situações, o algoritmo pode superar métodos comparativos tradicionais, pois evita operações de comparação entre elementos.

Exemplos de aplicações incluem:

- ordenação de notas de estudantes;

- classificação por idade;
- contagem de frequência de caracteres;
- processamento de dados estatísticos;
- algoritmos auxiliares como Radix Sort.

Devido à sua complexidade linear, o Counting Sort é particularmente eficiente para grandes quantidades de dados com domínio reduzido.

3.2 - LIMITAÇÕES DO ALGORITMO

Apesar de suas vantagens, o Counting Sort possui limitações importantes.

A principal delas está relacionada ao consumo de memória. Como o algoritmo necessita criar um vetor auxiliar proporcional ao maior valor presente na entrada, seu custo espacial pode tornar-se inviável quando o intervalo dos dados é muito grande.

Por exemplo, ordenar poucos elementos com valores extremamente altos pode resultar em desperdício significativo de memória.

Além disso, o algoritmo:

- não é adequado para dados contínuos ou números reais;
- apresenta baixa eficiência para intervalos extensos;
- depende do conhecimento prévio do domínio dos valores;
- pode consumir mais memória que algoritmos comparativos tradicionais.

Por essas razões, o Counting Sort é mais indicado para problemas específicos nos quais suas características sejam vantajosas.

4 - METODOLOGIA EXPERIMENTAL

Com o objetivo de validar experimentalmente a análise teórica do Counting Sort, foram desenvolvidas duas implementações do algoritmo: uma utilizando a linguagem Python e outra utilizando a linguagem Rust. A comparação entre as duas versões permitiu avaliar tanto a aderência dos resultados à complexidade teórica quanto o impacto das características de cada linguagem sobre o desempenho observado.

Para garantir a comparabilidade dos experimentos, ambas as implementações utilizaram exatamente os mesmos conjuntos de entrada. Os vetores foram gerados a partir de sementes previamente definidas, assegurando que Python e Rust processassem dados idênticos em todas as execuções.

Os experimentos foram realizados mantendo constante o número de elementos do vetor ($n = 1.000.000$) e variando apenas o intervalo dos valores possíveis (k). Foram definidos três cenários de teste:

- Cenário 1: $k = 100$
- Cenário 2: $k = 1.000.000$
- Cenário 3: $k = 100.000.000$

Cada cenário foi executado 30 vezes em ambas as linguagens. A partir dessas execuções, foram calculadas as médias dos tempos observados e os respectivos desvios-padrão, reduzindo a influência de oscilações ocasionais do sistema operacional e de outros fatores externos.

Como os experimentos foram executados em um ambiente real de sistema operacional, pequenas variações nos tempos medidos podem ocorrer devido a processos em segundo plano, gerenciamento de memória, escalonamento de tarefas e variações na frequência do processador.

Essas oscilações são esperadas em medições práticas de desempenho e devem ser consideradas na interpretação dos resultados.

A medição do tempo de execução foi realizada utilizando mecanismos de alta precisão disponibilizados pelas linguagens, permitindo obter resultados confiáveis para a análise de desempenho.

Como o Counting Sort apresenta complexidade $\Theta(n + k)$ independentemente da ordem dos elementos da entrada, os cenários foram definidos variando o valor de k , que representa o intervalo dos valores possíveis. Dessa forma, os experimentos buscaram avaliar o impacto do crescimento de k sobre o comportamento do algoritmo e verificar sua aderência à complexidade teórica prevista.

Os valores de k foram escolhidos para representar diferentes relações entre o número de elementos da entrada (n) e o intervalo dos valores possíveis. O primeiro cenário ($k = 100$) representa uma situação em que k é significativamente menor que n . O segundo cenário ($k = 1.000.000$) apresenta valores da mesma ordem de grandeza, enquanto o terceiro cenário ($k = 100.000.000$) representa uma situação em que k é muito superior a n . Essa configuração permite observar experimentalmente a influência do termo k na complexidade $\Theta(n + k)$.

4.1 - AMBIENTE DE EXECUÇÃO

Os experimentos foram realizados em um computador com as seguintes configurações:

- Processador: 3.89 GHz AMD64
- Memória RAM: 15.37 GB
- Sistema Operacional: Windows-11-10.0.26200-SP0
- Versão do Python: 3.13.7
- Versão do Rust: 1.95.0

Durante os testes, buscou-se minimizar interferências externas por meio do fechamento de aplicações não essenciais e da execução repetida dos experimentos, reduzindo a influência de processos concorrentes sobre as medições realizadas.

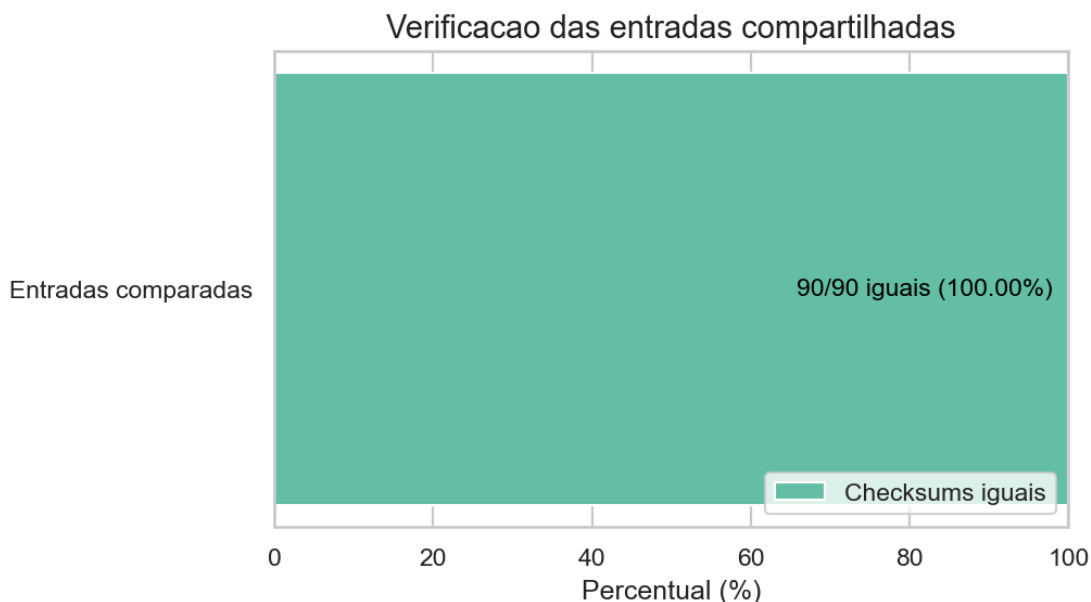
5 - RESULTADOS EXPERIMENTAIS

5.1 - VERIFICAÇÃO DA CORRETEDE DAS IMPLEMENTAÇÕES

Antes da análise de desempenho, foi realizada uma etapa de validação para garantir que as implementações em Python e Rust processassem as mesmas entradas. Para isso, foram calculados checksums sobre os vetores de entrada gerados deterministicamente a partir das mesmas sementes (seeds), antes da execução do algoritmo de ordenação.

Conforme apresentado na Figura abaixo, todas as verificações realizadas apresentaram concordância total entre as duas implementações. Esse resultado demonstra que ambas executam corretamente o algoritmo Counting Sort e produzem saídas idênticas para os mesmos conjuntos de entrada.

A validação da corretude é fundamental para a confiabilidade dos experimentos, pois garante que diferenças observadas posteriormente nos tempos de execução estejam relacionadas exclusivamente ao desempenho das linguagens utilizadas e não a inconsistências nos resultados produzidos.



Antes da análise detalhada dos resultados, a Tabela 1 apresenta um resumo dos principais valores obtidos nos experimentos realizados com as implementações em Python e Rust. Os dados incluem os tempos médios de execução e o speedup observado para cada cenário analisado.

Cenário	Python (s)	Rust (s)	Speedup
k = 100	0,0619	0,0014	44,9x
k = 1.000.000	0,2112	0,0119	17,8x
k = 100.000.000	9,8013	0,3085	31,8x

Tabela 1 – Resumo dos resultados experimentais

Pequenas variações entre execuções são esperadas devido à carga do sistema operacional, ao gerenciamento de memória, à frequência do processador e aos processos em segundo plano. Por isso, cada cenário foi executado 30 vezes e os resultados foram analisados pela média e pelo desvio padrão.

5.2 - COMPARAÇÃO DOS TEMPOS DE EXECUÇÃO

A Figura X apresenta os tempos médios de execução observados para as implementações em Python e Rust nos três cenários analisados. Em todos os experimentos foi mantido constante

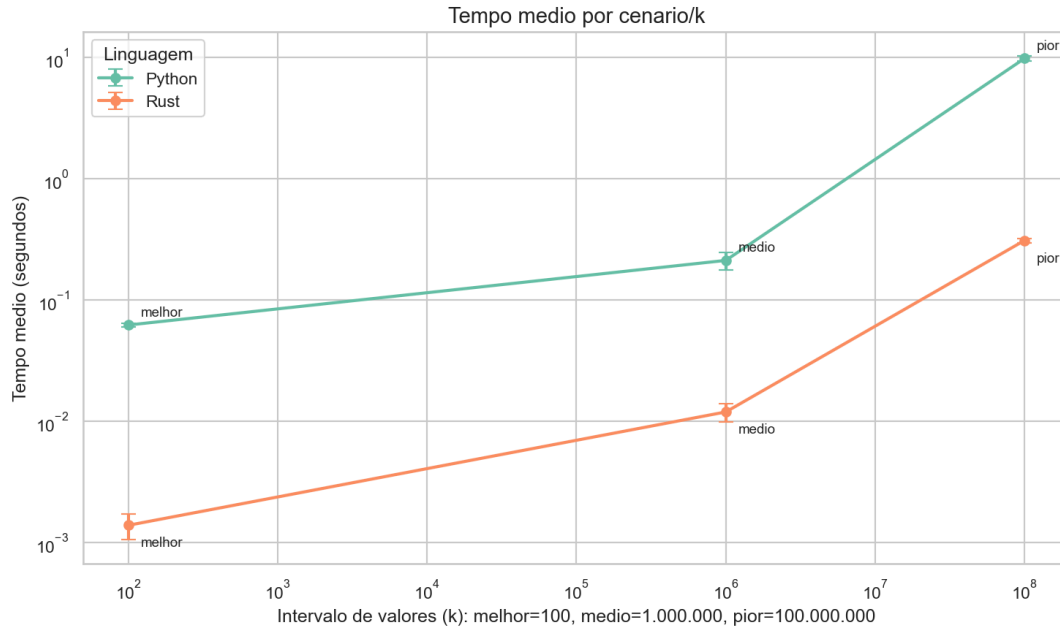
o número de elementos do vetor ($n = 1.000.000$), variando apenas o intervalo dos valores possíveis (k).

Os resultados evidenciam que o aumento de k provoca crescimento significativo do tempo de execução em ambas as implementações. No cenário mais favorável ($k = 100$), a implementação em Python apresentou tempo médio de aproximadamente 0,0619 segundos, enquanto a implementação em Rust executou em cerca de 0,0014 segundos. Quando o intervalo foi ampliado para $k = 1.000.000$, os tempos médios passaram para aproximadamente 0,2112 segundos em Python e 0,0119 segundos em Rust.

No cenário mais extremo ($k = 100.000.000$), observou-se o maior impacto do crescimento de k . A implementação em Python atingiu aproximadamente 9,8013 segundos de execução, enquanto a implementação em Rust executou a mesma tarefa em cerca de 0,3085 segundos.

Esse comportamento era esperado, uma vez que a complexidade do Counting Sort é dada por $\Theta(n + k)$, tornando o tamanho do vetor de contagem um fator determinante para o desempenho do algoritmo. Como o número de elementos permaneceu constante em todos os experimentos, o crescimento observado está diretamente relacionado ao aumento do intervalo dos valores possíveis.

Além disso, as barras de erro apresentaram baixa amplitude em relação às médias registradas, indicando estabilidade das medições e boa reprodutibilidade dos experimentos.



5.3 - ADERÊNCIA DOS RESULTADOS À COMPLEXIDADE TEÓRICA

Com o objetivo de verificar se o comportamento observado experimentalmente está de acordo com a análise assintótica do algoritmo, os tempos medidos foram comparados com a curva teórica esperada para o Counting Sort.

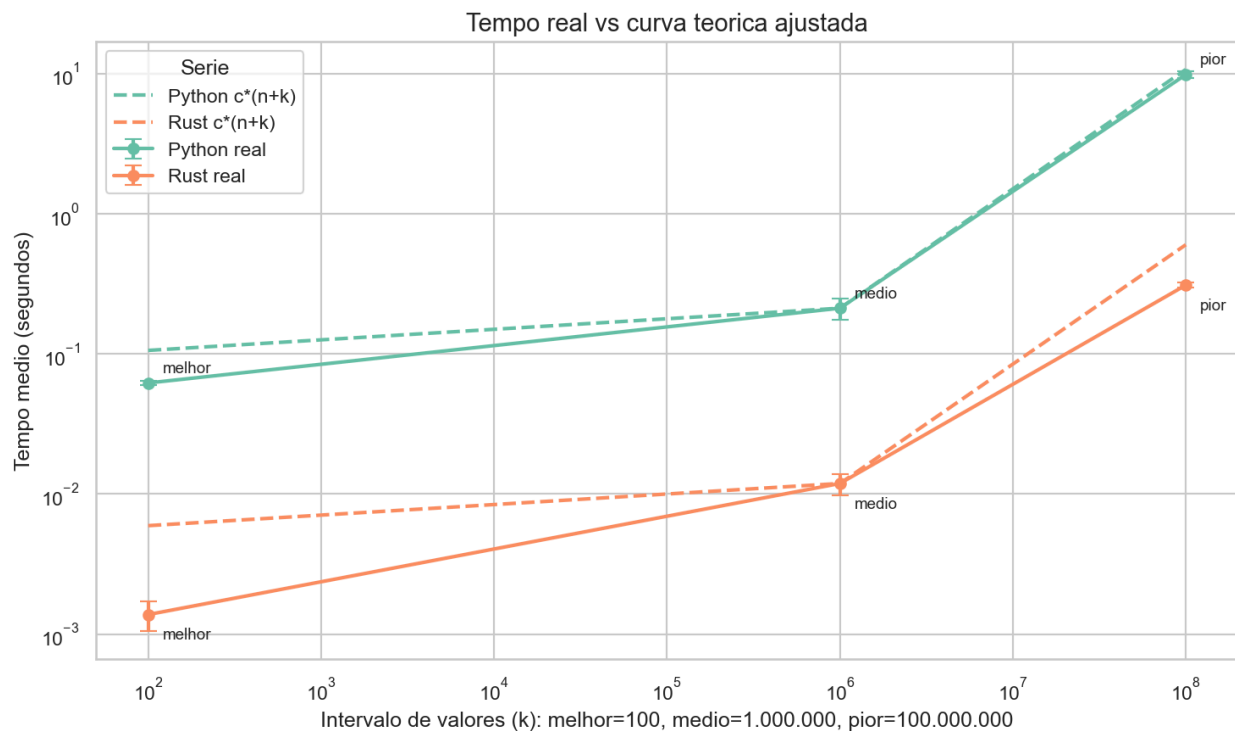
Os resultados experimentais demonstram forte aderência à complexidade $\Theta(n + k)$. Como o número de elementos permaneceu constante em todos os experimentos, o crescimento do tempo de execução foi influenciado principalmente pelo aumento do intervalo dos valores possíveis, exatamente como previsto pelo modelo teórico.

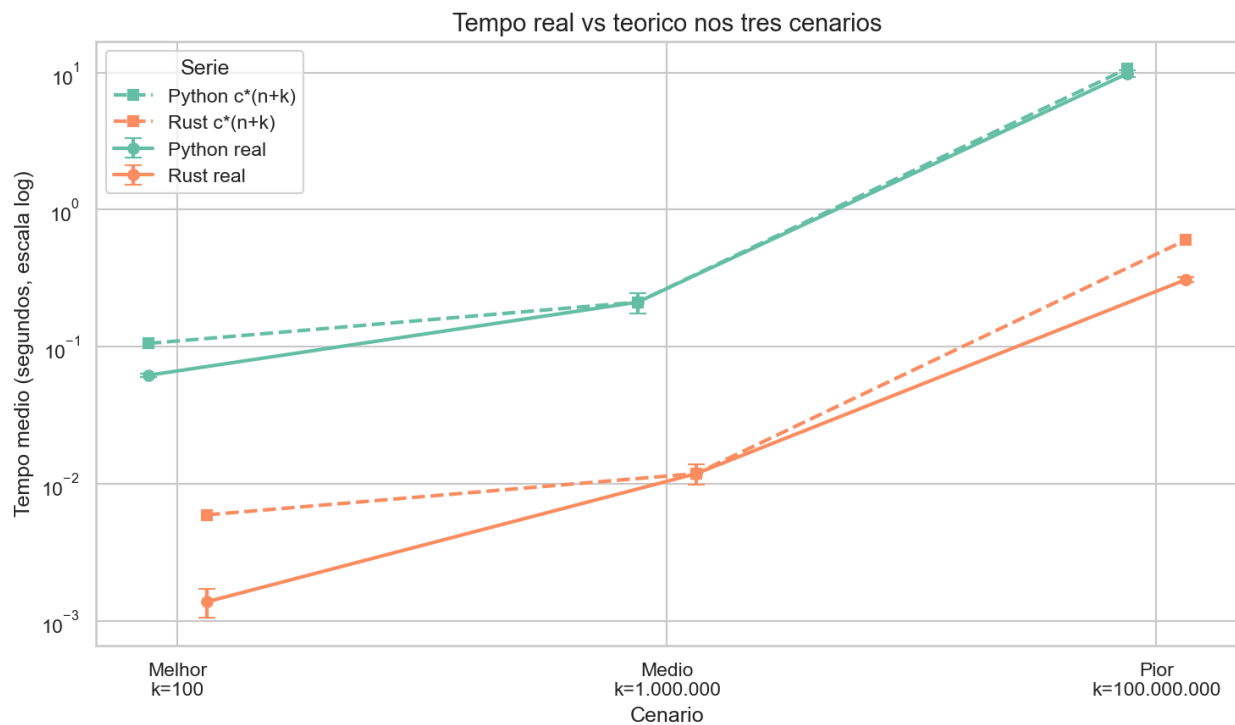
A comparação entre os dados reais e a curva ajustada mostra que ambas apresentam comportamento semelhante ao longo dos diferentes cenários analisados. Esse resultado reforça a validade da análise assintótica apresentada anteriormente e demonstra que o desempenho do algoritmo é diretamente impactado pelo tamanho do vetor de contagem utilizado internamente.

Os experimentos também evidenciam uma das principais limitações práticas do Counting Sort. Embora o algoritmo apresente crescimento linear, o aumento excessivo de k resulta em

maior consumo de memória e em aumento do tempo necessário para inicializar e percorrer o vetor de contagem. Dessa forma, mesmo mantendo a complexidade linear, o algoritmo pode tornar-se menos eficiente quando o intervalo dos valores possíveis é muito superior à quantidade de elementos efetivamente armazenados.

As pequenas diferenças observadas entre os resultados experimentais e a curva teórica são esperadas em ambientes reais de execução e podem ser atribuídas a fatores como gerenciamento de memória, utilização de cache, escalonamento do sistema operacional e otimizações específicas das linguagens utilizadas.





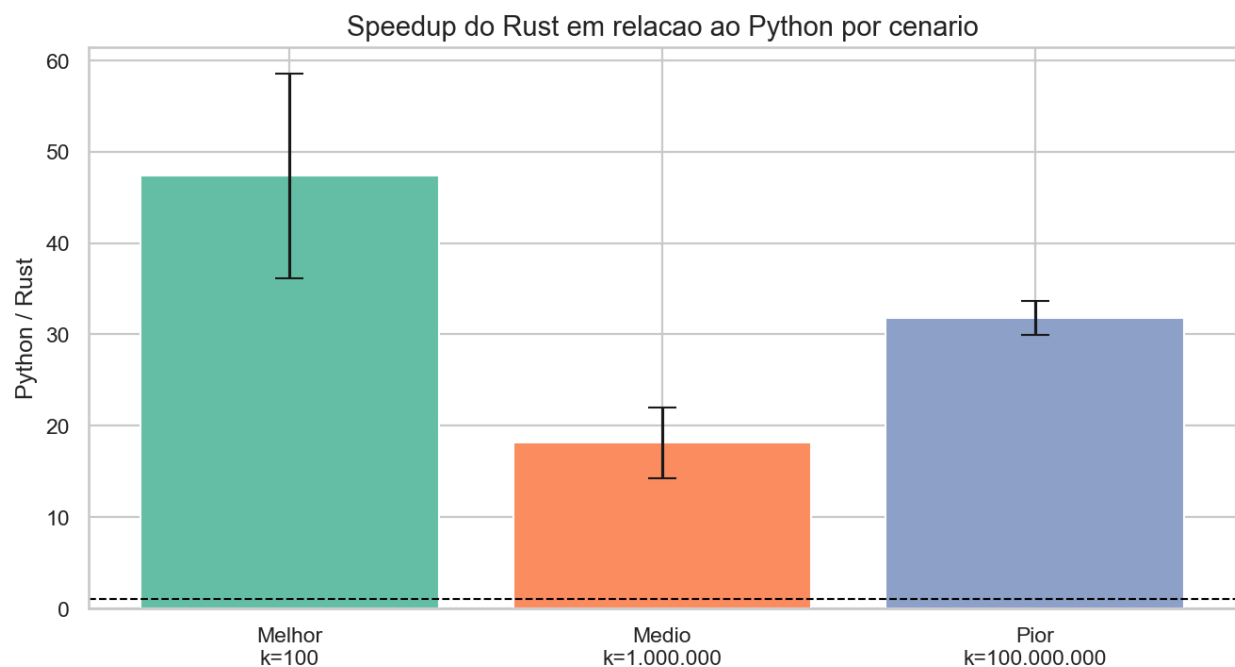
5.4 - COMPARAÇÃO ENTRE PYTHON E RUST

Além da comparação direta dos tempos de execução, foi calculado o speedup obtido pela implementação em Rust em relação à implementação em Python.

Os resultados mostram vantagem consistente para a linguagem Rust em todos os cenários analisados. No cenário com $k = 100$, a implementação em Rust foi aproximadamente 44,9 vezes mais rápida que a implementação em Python. Para $k = 1.000.000$, o speedup observado foi de aproximadamente 17,8 vezes. Já no cenário mais extremo, com $k = 100.000.000$, o ganho voltou a crescer, alcançando aproximadamente 31,8 vezes.

Esse comportamento pode ser explicado pelas diferenças entre os modelos de execução das linguagens. Enquanto o Python é interpretado e possui maior sobrecarga associada ao gerenciamento de objetos e execução dinâmica, o Rust é compilado para código nativo e permite otimizações de baixo nível realizadas pelo compilador.

Os resultados demonstram que, embora ambas as implementações apresentem o mesmo comportamento assintótico, fatores relacionados à implementação influenciam significativamente o desempenho prático do algoritmo. Esse resultado evidencia que a análise assintótica, embora fundamental para compreender o crescimento de um algoritmo, não é suficiente para prever seu desempenho real. Mesmo apresentando a mesma complexidade teórica, implementações distintas podem produzir tempos de execução significativamente diferentes devido a fatores como otimizações do compilador, gerenciamento de memória, representação de dados e características da linguagem utilizada.



5.5 - TAXA DE PROCESSAMENTO

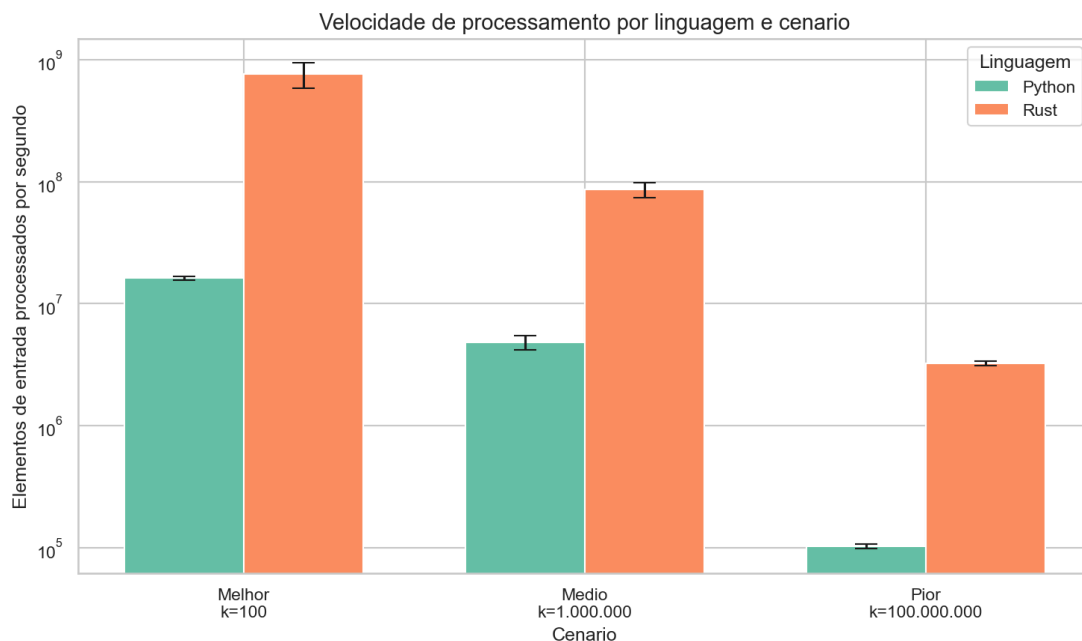
A taxa de processamento foi calculada com o objetivo de avaliar a quantidade de elementos processados por segundo em cada linguagem e cenário experimental.

No cenário mais favorável ($k = 100$), a implementação em Python processou aproximadamente 16,2 milhões de elementos por segundo, enquanto a implementação em Rust atingiu cerca de 766 milhões de elementos por segundo. Para $k = 1.000.000$, as taxas observadas foram aproximadamente 4,84 milhões de elementos por segundo em Python e 86,2 milhões de elementos por segundo em Rust.

No cenário mais desfavorável ($k = 100.000.000$), a taxa de processamento caiu para aproximadamente 102 mil elementos por segundo em Python, enquanto a implementação em Rust manteve desempenho próximo de 3,25 milhões de elementos por segundo.

Os resultados indicam superioridade consistente da implementação em Rust, que apresentou capacidade significativamente maior de processamento em todos os cenários avaliados. Mesmo quando o intervalo dos valores possíveis aumentou consideravelmente, a implementação em Rust manteve taxas de processamento superiores às observadas em Python.

Observa-se também que a taxa de processamento diminui à medida que o valor de k aumenta. Esse comportamento reforça a influência do vetor de contagem sobre o desempenho do algoritmo, evidenciando que a eficiência prática do Counting Sort está diretamente relacionada à relação entre o número de elementos processados e o intervalo dos valores possíveis.



6 - REFLEXÃO FINAL SOBRE COMPLEXIDADE

O Counting Sort pertence à classe P, pois resolve o problema de ordenação em tempo polinomial. Sua complexidade temporal é $\Theta(n + k)$, o que garante que o tempo de execução cresce de forma previsível em função do tamanho da entrada e do intervalo dos valores possíveis. Dessa forma, o algoritmo pode ser executado de maneira eficiente mesmo para grandes volumes de dados, desde que o valor de k permaneça controlado.

Não existe uma formulação conhecida do problema resolvido pelo Counting Sort que pertença à classe NP-completa, uma vez que a ordenação já possui soluções eficientes em tempo polinomial.

Entretanto, existem problemas relacionados à organização e ao processamento de dados que pertencem à classe NP ou são NP-completos. Exemplos incluem o Problema da Mochila (Knapsack), o Problema da Soma de Subconjuntos (Subset Sum) e o Problema do Caixeiro Viajante (Travelling Salesman Problem). Diferentemente desses problemas, o Counting Sort não exige busca combinatória nem crescimento exponencial do número de operações, sendo considerado um algoritmo eficiente do ponto de vista computacional.

7 - CONCLUSÃO

O presente trabalho teve como objetivo analisar teoricamente e experimentalmente o algoritmo Counting Sort, comparando implementações desenvolvidas nas linguagens Python e Rust. A partir da análise assintótica, verificou-se que o algoritmo apresenta complexidade temporal $\Theta(n + k)$ e complexidade espacial $O(n + k)$, características que o tornam uma alternativa eficiente para a ordenação de conjuntos de números inteiros quando o intervalo dos valores possíveis permanece relativamente pequeno.

Os experimentos realizados confirmaram as previsões teóricas. Observou-se que o aumento do valor de k impacta diretamente o tempo de execução, evidenciando a influência do vetor de contagem sobre o desempenho do algoritmo. Os gráficos produzidos demonstraram forte aderência entre os resultados experimentais e a curva teórica esperada, validando a análise de complexidade apresentada.

A comparação entre as implementações mostrou vantagem consistente para a linguagem Rust em todos os cenários avaliados. Embora ambas apresentem o mesmo comportamento assintótico, fatores relacionados à compilação nativa, otimização de código e gerenciamento de memória resultaram em desempenho significativamente superior para a implementação em Rust. Os experimentos mostraram ganhos de desempenho entre aproximadamente 17,8 e 44,9 vezes para a implementação em Rust, dependendo do cenário analisado.

Por fim, conclui-se que o Counting Sort é uma solução eficiente para problemas específicos de ordenação, especialmente quando o intervalo dos valores possíveis não é excessivamente grande. Apesar de suas limitações de memória em cenários com valores de k muito elevados, o algoritmo demonstrou desempenho previsível e aderente à teoria, atendendo aos objetivos propostos neste trabalho.